

FICHE DE TD 3/3 • DIFFICILE

Informatique • 3ème

Utilisation des structures de contrôle

Informatique – classe de 3ème

Les exercices suivants mobilisent l'écriture algorithmique, le tracé d'algorigrammes et l'analyse critique de boucles et conditions imbriquées. Toute variable est typée implicitement (Entier, Réel, Booléen, Chaîne).

EXERCICE 1 Système de billetterie dynamique

Un cinéma automatise sa caisse. Le prix de base est 2500 FCFA. Des réductions s'appliquent cumulativement : -15 % pour les étudiants (sur présentation de carte), -10 % pour les séances avant 14h, -5 % pour les détenteurs de la carte fidélité. Le programme gère une file d'attente de N clients ($N > 0$). Pour chaque client, on saisit : `estEtudiant` (Booléen), `heureSeance` (Entier, format 24h), `aCarteFidelite` (Booléen). Le programme affiche le prix final arrondi à l'entier inférieur et cumule le chiffre d'affaires total.

1. Écrire l'algorithme complet (variables, saisie N , boucle principale, calculs, affichages). Utiliser une structure `Pour` pour la file d'attente.
2. Dessiner l'algorigramme correspondant uniquement au bloc de calcul du prix pour un client (structures alternatives imbriquées ou successives).
3. Modifier l'algorithme pour qu'il s'arrête prématurément si le chiffre d'affaires dépasse 500000 FCFA (utiliser `TantQue` avec condition composée). Justifier le changement de structure itérative.
4. Proposer une modification pour gérer le cas où un client achète k places identiques ($k \geq 1$ saisi au clavier) sans dupliquer le code de calcul.

EXERCICE 2 Analyse et correction d'un algorithme de tri

On considère l'algorithme suivant, sensé trier un tableau $T[0..n-1]$ d'entiers par ordre croissant (Tri à bulles optimisé) :

Algorithme `TriBullesBizarre`

Variables : T : Tableau[0.. $n-1$] d'Entiers; $n, i, j, temp$: Entier; `echange` : Booléen

Debut

`echange` <- Vrai

i <- 0

 TantQue ($i < n-1$) ET (`echange` = Vrai) Faire

`echange` <- Faux

 Pour j <- 0 jusqu'à $n-2$ Faire

 Si $T[j] < T[j+1]$ Alors

```

        temp <- T[j]
        T[j] <- T[j+1]
        T[j+1] <- temp
        echange <- Vrai
    FinSi
FinPour
    i <- i + 1
FinTantQue
Fin

```

1. Identifier trois erreurs logiques ou syntaxiques majeures empêchant le tri correct (ordre croissant, borne de boucle, condition d'arrêt). Les lister sans réécrire le code.
2. Pour $n = 5$ et $T = [3, 1, 4, 1, 5]$, effectuer la trace complète de la première itération de la boucle TantQue (valeurs de j , $T[j]$, $T[j+1]$, contenu de T après chaque Si vrai, valeur finale de echange).
3. Écrire la version corrigée et optimisée de l'algorithme (borne interne dégressive, bon sens de comparaison).
4. Démontrer par récurrence sur i que, à la fin de l'itération i de la boucle externe corrigée, les $i + 1$ plus grands éléments sont à leurs positions finales et triés.

EXERCICE 3 Chiffrement de Vigenère – Synthèse

On veut implémenter le chiffrement de Vigenère (clé répétée) sur des majuscules A-Z (code ASCII 65–90). La clé K est une chaîne de longueur L ($1 \leq L \leq 20$). Le message M a une longueur N ($N \geq 1$). Caractère chiffré : $C_i = (M_i + K_{i \bmod L}) \bmod 26$.

1. Écrire une fonction `Code(c : Caractere) : Entier` renvoyant la position 0–25 (A=0) ou -1 si hors intervalle.
2. Écrire la procédure `Chiffrer(M, K : Chaîne) : Chaîne` renvoyant le cryptogramme. Gérer les caractères non alphabétiques (copie inchangée, ne consomment pas de lettre de clé). Utiliser `Pour i <- 0 jusqu'à Longueur(M)-1`.
3. Même question avec une boucle TantQue gérant un index explicite. Comparer la lisibilité des deux versions pour ce problème précis.
4. On veut maintenant le déchiffrement $M_i = (C_i - K_{i \bmod L} + 26) \bmod 26$. Proposer une factorisation du code (procédure unique paramétrée par un sens $s \in \{-1, +1\}$) et écrire l'appel pour déchiffrer.
5. Analyser la complexité en temps et en espace mémoire de votre procédure factorisée en fonction de N et L . Justifier.

EXERCICE 4 Jeu de Nim variant – Conception et preuve

Deux joueurs retirent alternativement 1, 2 ou 3 allumettes d'un tas initial de N allumettes ($N \geq 1$). Celui qui prend la dernière perd (version misère). On veut un algorithme `Jouer(N)` qui simule une partie complète contre un joueur "parfait" (stratégie gagnante si possible, sinon coup aléatoire légal). L'humain joue en premier.

1. Définir la condition de position gagnante/perdante (PG/PP) pour ce jeu misère (indice : analyser $N = 1, 2, 3, 4, 5 \dots$).
2. Écrire l'algorithme principal `Jouer(N)` : boucle de jeu, affichage de l'état, saisie sécurisée du coup humain (contrôle $1 \leq coup \leq 3$ et $coup \leq reste$), coup de l'ordinateur (stratégie : aller en position PP si possible, sinon 1), détection fin de partie, annonce du vainqueur.
3. Dessiner l'algorithme de la fonction `CoupOrdi(reste : Entier) : Entier` implémentant la stratégie parfaite.
4. Prouver que si $N \equiv 1 \pmod{4}$ au départ, le premier joueur (humain) perd contre l'ordinateur parfait, quel que soit son jeu.
5. Modifier l'algorithme pour permettre une partie "Meilleur de 3 manches" avec alternance du joueur qui commence. Variable `scoreHumain`, `scoreOrdi`.